République du Sénégal

un peuple - un but - une foi

Ministère de la Formation Professionnelle Technique

<u>THÈME</u> : Gestion d'un studio de fitness — FITNESS 221

<u>Apprenant</u>: Serigne Mbaye THIAM

1. Introduction

Le secteur du sport et du bien-être connaît aujourd'hui une croissance sans précédent. Pourtant, de nombreux studios de fitness gèrent encore leurs réservations et leurs plannings de manière artisanale, ce qui entraîne des erreurs de surréservation et une frustration tant pour les coachs que pour les clients.

C'est pour répondre à ce défi que j'ai développé Fitness 221. Plus qu'une simple application de gestion, c'est une solution robuste qui automatise le cycle de vie complet d'un studio : de l'authentification sécurisée des membres à la validation en temps réel des capacités de séances.

Contexte

À l'ère de la transformation numérique, la gestion manuelle des plannings sportifs devient un frein majeur pour la rentabilité et l'image de marque des centres de fitness. Le projet Fitness 221 naît de l'observation d'un besoin critique : offrir une expérience utilisateur fluide tout en garantissant une gestion administrative sans faille.

Problématique & Solution

Comment garantir qu'un coach n'est pas surchargé ou qu'une salle ne dépasse pas sa capacité d'accueil à 10h du matin ? La solution apportée par Fitness 221 repose sur trois piliers :

1. L'intelligence logicielle : Des algorithmes de validation qui contrôlent chaque réservation à la milliseconde près.
2. La sécurité absolue : Une protection des données personnelles via des standards de chiffrement industriels (JWT).
3. La résilience infra : Une architecture Dockerisée permettant un redéploiement instantané.
4. Utilisation d’un outil permettant d’exposer le HTTP en HTTPS.

Périmètre Technique

Ce projet met en œuvre un écosystème complet :

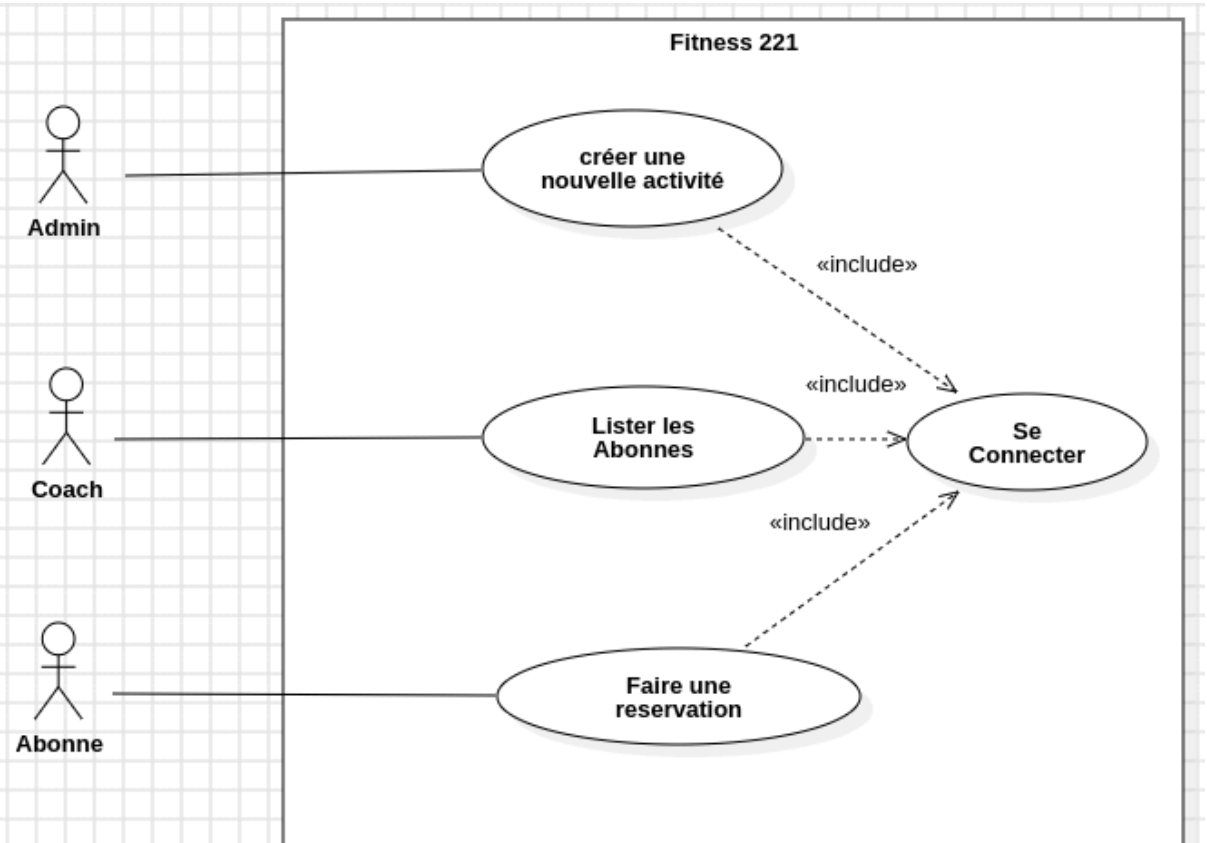
- Backend : Node.js & Express pour la vélocité.
- Data : PostgreSQL & Prisma pour la robustesse des données.
- DevOps : Dokploy & Cloudflare Tunnel pour une accessibilité sécurisée.

2. Analyse Fonctionnelle

Fonctionnalités par Profil

- Authentification et Profils
  - Inscription & Connexion : Inscription d'utilisateurs avec rôles (ADMIN, COACH, ABONNE).
  - Protection JWT : Sécurisation de toutes les routes de l'API.
- Gestion des Entités (Admin)
  - Coachs : Création, modification et suppression des profils de coachs avec leurs spécialités.
  - Abonnés : Gestion des inscriptions d'abonnés et de leurs types d'abonnement (MENSUEL, TRIMESTRIEL, ANNUEL).
  - Activités : Définition des séances de fitness avec code, nom, durée et capacité maximale (placesMax).

- Système de Réservation (Cœur du projet)
    - Réservation de séances : Un abonné peut s'inscrire à une séance supervisée par un coach.
    - Règles Métier (Validations) :
        - Disponibilité du Coach : Un coach ne peut pas avoir deux réservations sur le même créneau horaire.
        - Capacité de l'activité : On ne peut pas dépasser le nombre de places maximum (placesMax) pour une séance donnée.
        - Unicité de l'abonné : Un abonné ne peut pas réserver deux fois la même activité sur le même créneau.
        - Contrôle Temporel : Les réservations ne sont autorisées que pour des dates futures.
- Cas d'Utilisation (Use Cases)
    - En tant qu'Admin : Je veux créer une nouvelle activité "Yoga" pour diversifier le studio.
    - En tant qu'Abonné : Je veux réserver ma place pour la séance de "Musculation" de demain à 10h.
    - En tant que Coach : Je veux voir la liste des abonnés inscrits à ma séance de ce soir.

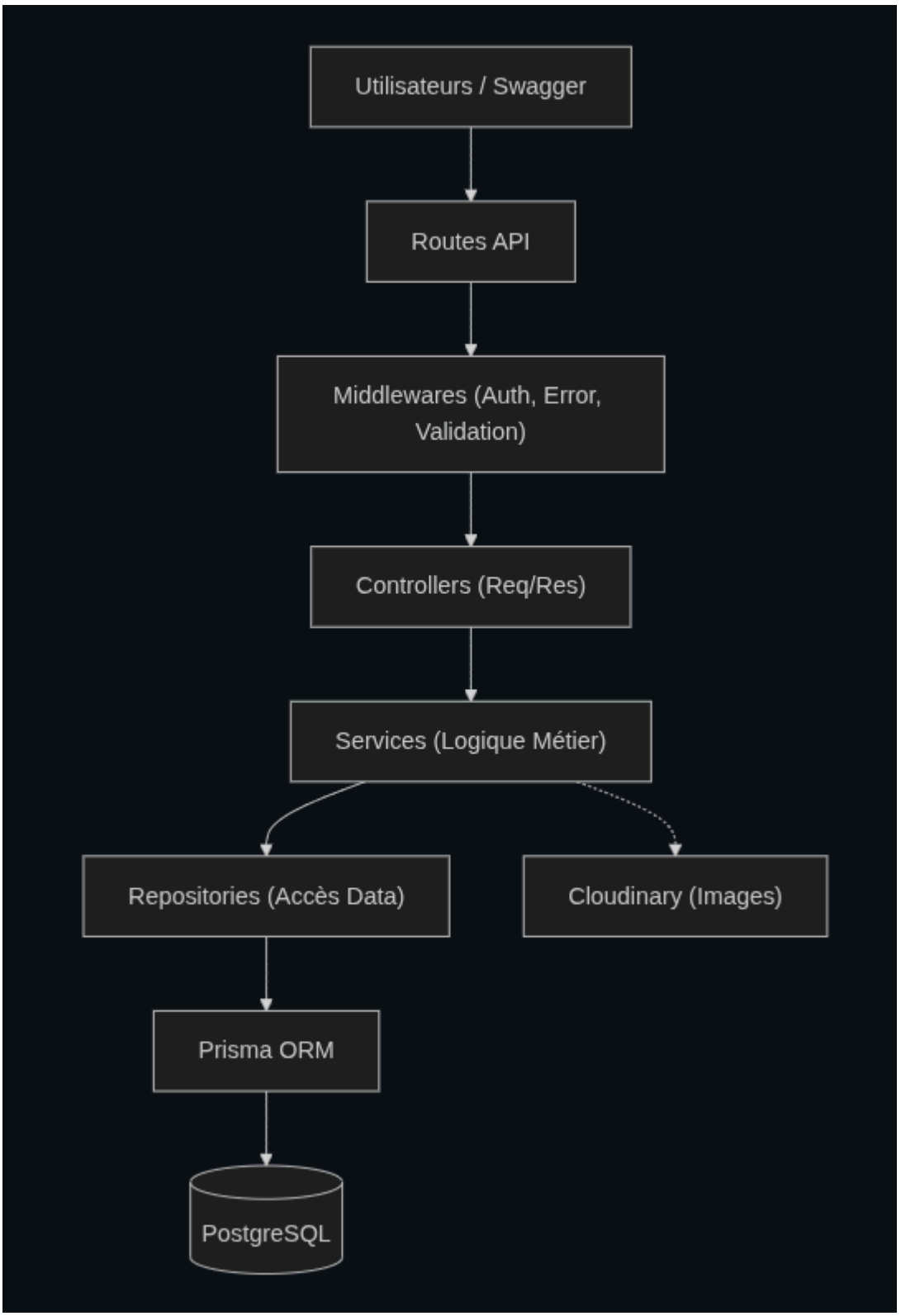


3. Architecture Logicielle

Architecture en Couches (Layered Architecture)

Le projet utilise une architecture multicouche pour séparer les responsabilités :

- Routes (src/routes) : Point d'entrée de l'API (Routage Express).
- Controllers (src/controllers) : Gestion des requêtes HTTP (Extraction des paramètres, Réponse).
- Services (src/services) : Logique métier complète (Règles de réservation, Calculs, Validations).
- Repositories (src/repositories) : Abstraction de l'accès aux données (Requêtes Prisma).
- Modèle (Prisma Client) : Interaction avec la base de données PostgreSQL.



Middlewares et Sécurité

- Authentification JWT

Le middleware auth vérifie chaque requête entrante. Il extrait le token, le valide et injecte l'expéditeur dans l'objet req.

- Gestion des Erreurs

Un middleware global errorHandler centralise toutes les exceptions de l'application, assurant que l'utilisateur reçoit toujours une réponse JSON cohérente et sécurisée.

- Validation Zod & Joi

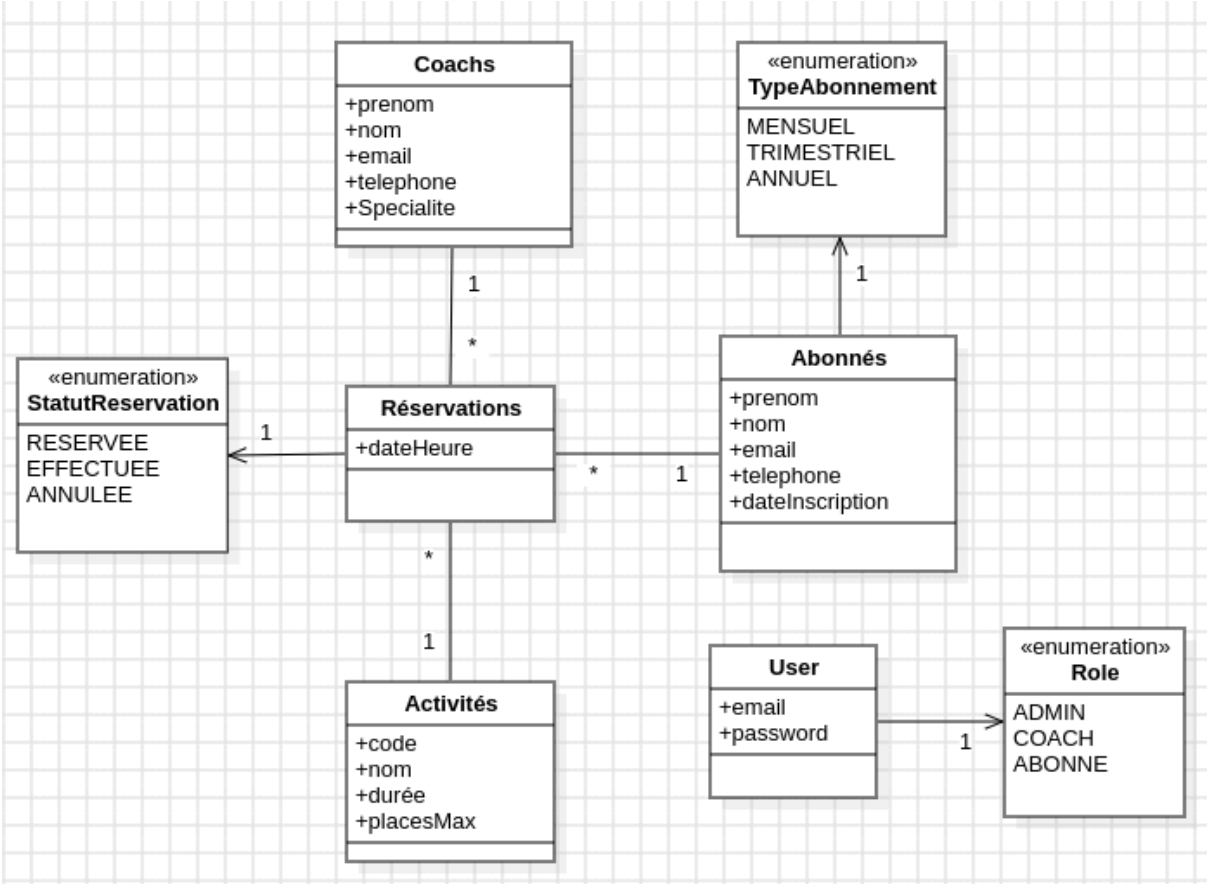
L'utilisation de schémas de validation garantit que seules des données conformes entrent dans le système, évitant ainsi les erreurs inattendues en base de données.

4. Conception des Données

Modèle de Données (Entities)

Le projet repose sur 4 entités principales interdépendantes :

- User : Identités, emails et rôles (ADMIN, COACH, ABONNE).
- Coach : Informations sur les instructeurs (Prénom, Nom, Spécialité).
- Abonné : Membres du studio avec leur type d'abonnement.
- Activité : Séances proposées (Yoga, Fitness, Musculation).
- Réservation : L'entité pivot liant un Abonné, un Coach et une Activité à une heure précise.



Prisma ORM

Prisma est utilisé comme ORM (Object-Relational Mapper) pour interagir avec la base de données PostgreSQL.

- Type Safety : Génération automatique d'un client TypeScript/JavaScript typé selon le schéma.
- Migrations : Gestion de l'évolution de la base de données avec npx prisma migrate.
- Relations : Gestion native des relations via les attributs @relation (ex: Reservation lie l'ID de l'abonné et de l'activité).

5. Infrastructure et Déploiement

Conteneurisation (Docker)

L'application est entièrement conteneurisée pour garantir un environnement de production identique au développement.

Dockerfile Multi-Stage

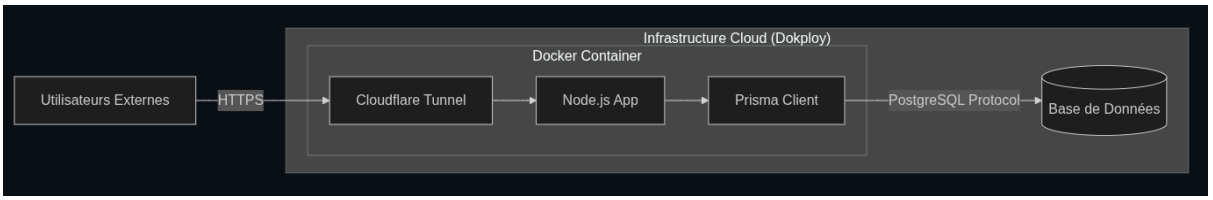
Nous utilisons une image node:20-slim pour assurer la légèreté et la compatibilité avec le moteur Prisma (OpenSSL inclus).

- Stage 1 (Build) : Installation de toutes les dépendances et génération du client Prisma.
- Stage 2 (Production) : Copie uniquement des fichiers nécessaires pour réduire la taille de l'image finale.

Déploiement sur Dokploy

Le projet est déployé en continu sur une instance Dokploy. Le cycle de vie est géré par un script entrypoint.sh :

- Migrations : Exécution automatique de npx prisma migrate deploy.
- Tunnel : Activation de cloudflared pour l'exposition publique.
- Démarrage : Lancement du serveur Node.js.



Exposition via Cloudflare Tunnel

Pour éviter d'exposer directement le serveur Dokploy et ses ports, nous utilisons un Cloudflare Tunnel. Cela permet :

- HTTPS automatique : Chiffrement de toutes les connexions.
- Zero Trust : Possibilité de restreindre l'accès à l'API.
- Sécurité : Aucun port ouvert directement sur le pare-feu du serveur.

Lien GITHUB: https://github.com/smt197/fitness-221.git